

TP1 - Build Your First App

Mobile & Ubiquitous Computing 2026/2027

Ciências, Departamento de Informática

Diogo Branco

Laboratory classes

Diogo Branco

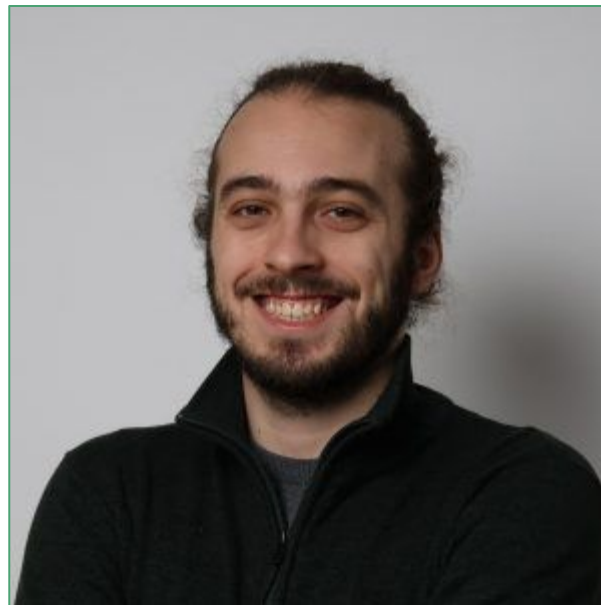
techandpeople.github.io

djbranco@ciencias.ulisboa.pt

Research

Digital Health Research, HCI, HDI,
Collaborative Health

[Diogo Branco - Google Scholar](#)



Planning – laboratory classes (tentative)

Part 1

Laboratory assignments

W01 • 16/Set

W02 • 23/Set

Intro, Android Studio, Kotlin & First Hands-on Lab

W03 • 30/Set

Android Basic GUI & Navigation • #1 Lab Assign (First App)

W04 • 07/Out

App Architecture & Getting Data from the Web Data • #2 Lab Assign (Multiple Screens App)

W05 • 14/Out

Persistent Data • #3 Lab Assign (API use, Data Fetch & Display)

W06 • 21/Out

No classes (Ciências Research Day)

W07 • 28/Out

Android Sensors • #4 Lab Assign (Remote Local Storage)

Part 2

Project Development

W08 • 04/Nov

Project Proposal

W09 • 11/Nov

Project - Low-Fidelity Prototyping • #5 Lab Assign (Using Sensors)

W10 • 18/Nov

Project - GUI

W11 • 25/Nov

Checkpoint 1

W12 • 02/Dez

Project – Sensors & DB

W13 • 09/Dez

Project – Multi-user & Services

W14 • 16/Dez

Checkpoint 2 – Alpha Version

FINAL • 20/Jan • PRESENTATION

Open Studio: Final Project Demo & Deliverables

Lab Assignments

- Individual assignments
 - Individual evaluation
- 5 Assignments
 - Training guides + exercises
- The process:
 - Training guides and samples
To be consumed in class
 - The exercises
Launched in a class and evaluated in the next one
- Evaluation
 - The implementation, demonstration, and knowledge contribute to the grade (demo + code)
- Grading
 - Failing to demonstrate corresponds to a zero grade
 - No second chances
 - The 5 assignments correspond to **15%** of the course classification

Project

- Objective

- Build an App or a set of Apps

- Basic requirements

- Good user experience

- Feature rich (service integration, ..)

- Good data management

- Distributed (multi-user)

- Adequate combination of the above or others

- More requirements

- Creativity, arguments, design planning

- Bibliography, writing and structure

- Fail to mention bibliography could be considered fraud

- Group size

- 3 Students

Available mobile equipment & Lab software conditions

Project

Themes

- **You propose**
- Has to be an app that only makes sense in a **fictional world** (e.g. Lord of the Rings)
 - *Some of its features need to only make sense in that context*
- The app can be on any topic (e.g. health, navigation, shopping, games, sports, lifestyle, business, education etc)

Project

Deliverables

Project Proposal Presentation

- It should motivate the concept, present related work, and describe the approach and features
- Submission: one day before presentation
- Presentation and Discussion: see CM Plan

Project final deliverables

- Project Report
 - 1 short paper, 2-5 page
 - ACM format (2 columns) & PDF
- Source code & App
- Promotional video (less than 2 min)

Checkpoints

- App presentation & demo
- Dates: see CM plan

Open Studio/demo

- Final Project demo
- “Oral exam” with teachers & external jury
- **20 January**

Evaluation

- Project proposal presentation + 2 checkpoints (**15%**)
 - Project final deliverable & Open Studio (**40%**)
- Minimum score: 9.5

Evaluation of working students

- Evaluation is similar to regular students, except:
 - Lab Assignments
 - A challenge is posted weekly
 - An email should be sent by next Wednesday at 23:59, with the link to the solution on GitHub and the APK that accomplishes the objectives
 - Project checkpoints
 - Presentation/demo of checkpoints can be pre-recorded (video) and shared before class

Start thinking of Themes for Project

- Discuss them with me throughout the next classes
- Every app has to be different

Example (ChatGPT):

Quaffle Quest, the ultimate mobile app for witches and wizards to **track down missing** Quidditch balls! Using enchanted ball radar and **real-time alerts**, you'll never lose sight of the Quaffle, rogue Bludgers, or the elusive Golden Snitch—**on and off the pitch**. With advanced **features like Bludger detection, Snitch speed tracking**, and a **magical Lost Ball Recovery Charm, Quaffle Quest**.

Whether you're playing, **coaching**, or just **spectating**, this app is your essential Quidditch companion

Bid

- What you are fighting for:
 - A project you like
 - Material to work (smartphones, smartwatches, tablets, bracelets, Bluetooth beacons, robots)
- A Good proposal
 - Has those benefits
 - And a good grade
- A Bad proposal
 - Is entitled to a proposal with pre-defined requirements
 - Less motivation to work?

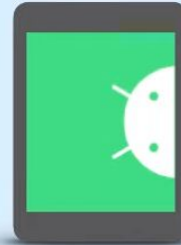
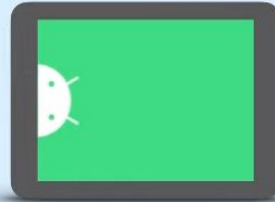
02



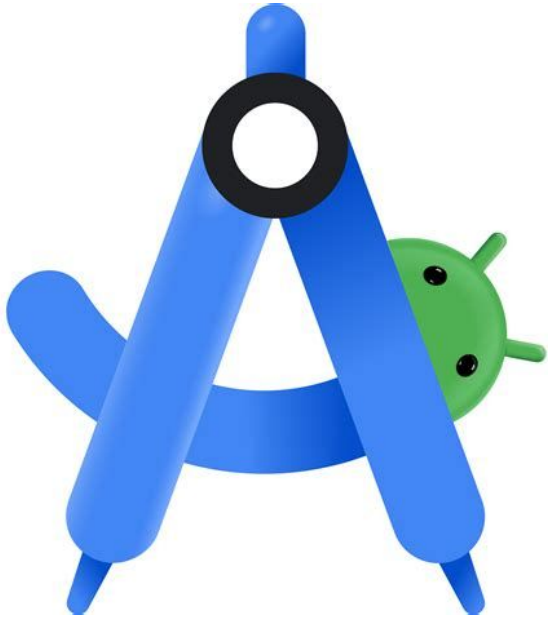
ANDROID BASICS

Android

2.5B Android devices



Android Studio



<https://developer.android.com/studio/intro>

Android Basics

- Multi-user Linux system in which each app is a different user
- Each process has its own virtual machine (VM),
 - app's code runs in isolation from other apps
- By default you have access to the bare minimum!
 - Sensors, services, etc

Android Basics - App components

Activities

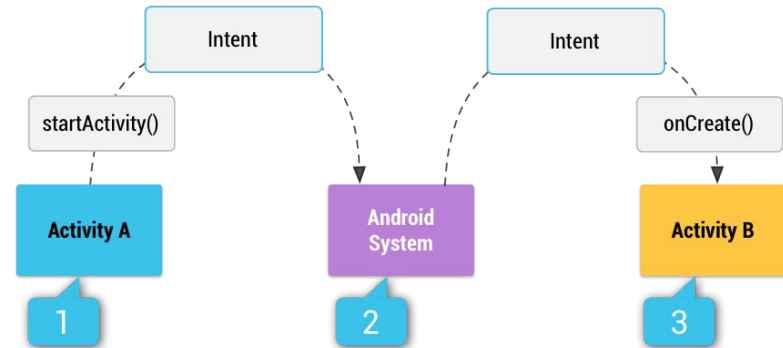
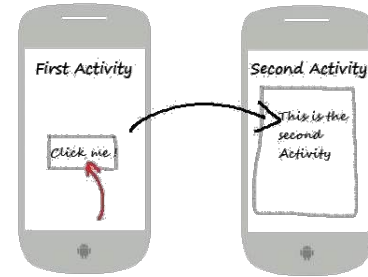
Services

Broadcast receivers

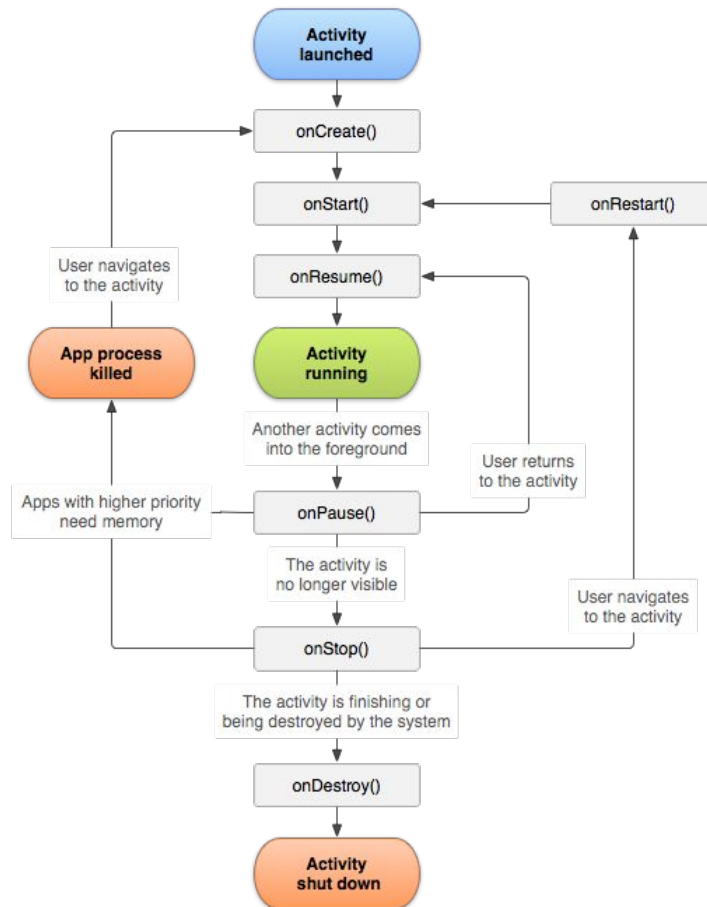
Content providers

Android Basics - Activities

- It represents a single screen with a user interface
 - Keeping track of what the user currently cares about—what is on-screen

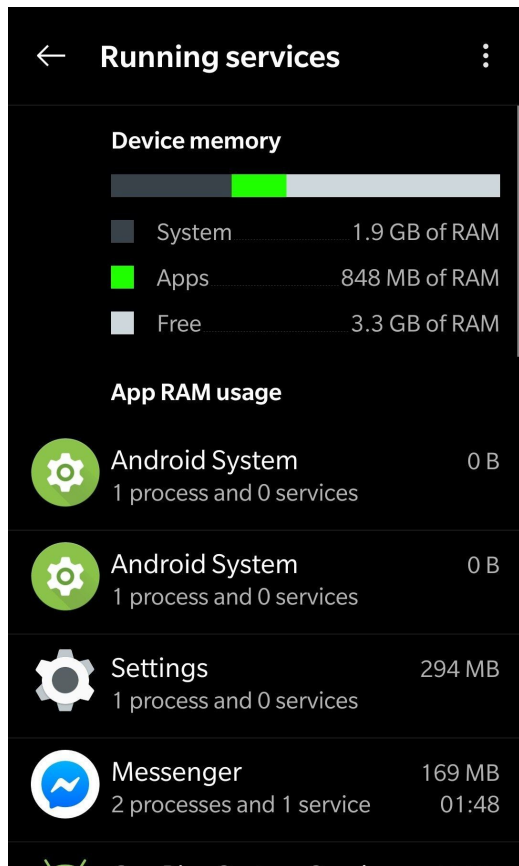


Android Basics - Activities



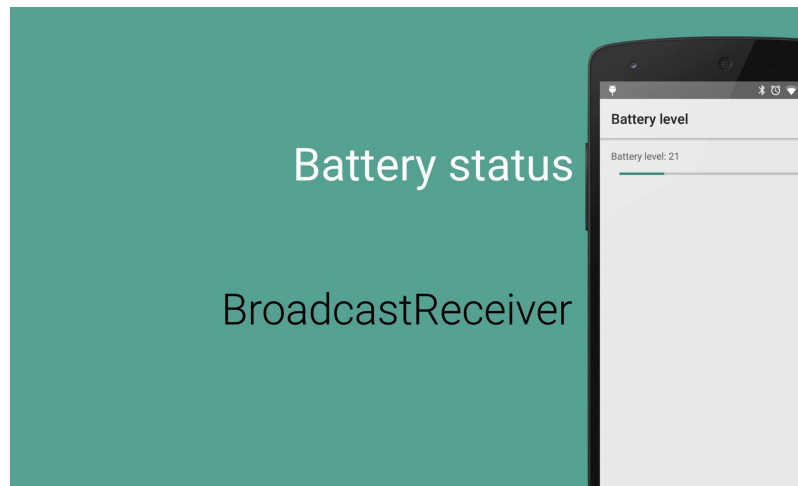
Android Basics - Services

- General-purpose entry point for keeping an app running in the background
 - A service does not provide a user interface



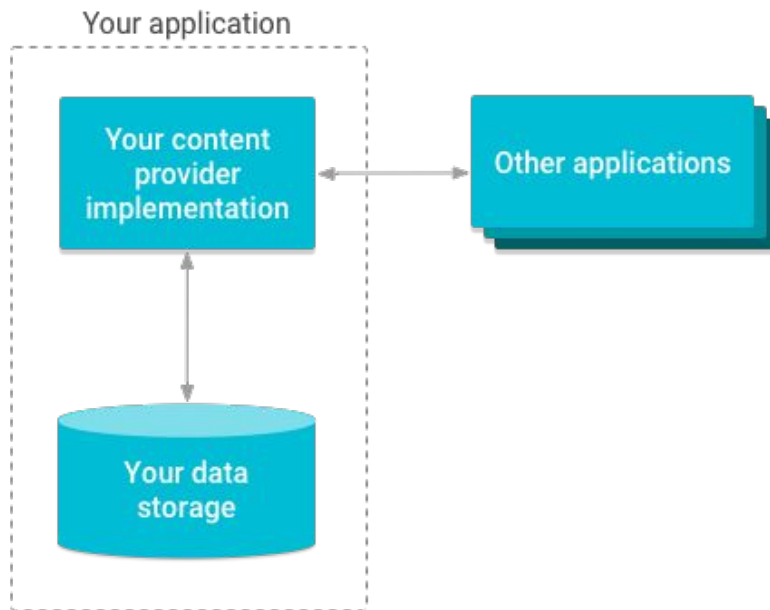
Android Basics - Broadcast receivers

- Lets the system deliver events to the app outside of a regular user flow so the app can respond to system-wide broadcast announcements
 - A broadcast receiver does not provide a user interface



Android Basics - Content providers

- Content provider manages a shared set of app data that you can store in the file system
 - SQLite database, on the web, or on any other persistent storage location that your app can access.
 - Other apps can query or modify the data, if the content provider permits it.
 - Example:
 - List of phone contacts
 - Photos storage



Android Basics - Event Based!

- **Intents (Asynchronous Messages)**
 - Activate and transition between activities, services, and broadcast receivers



Android Button Click

Click on the buttons to add some text on the screen below

My Button 1

My Button 2

Clicked on Button 2
bla bla bla ...

Android Basics

More Details:

<https://developer.android.com/guide/components/fundamentals?hl=en>

Develop Android apps with Kotlin

Write better Android apps faster with Kotlin. Kotlin is a modern statically typed programming language used by over 60% of professional Android developers that helps boost productivity, developer satisfaction, and code safety.

Get started



Expressive and concise

Kotlin's modern language features allow you to focus on expressing your ideas and write less boilerplate code.

[Learn more](#)



Safer code

With nullability included in its type system, Kotlin helps you avoid NullPointerExceptions. Android apps that use Kotlin are 20% less likely to crash.

[Learn more](#)



Jetpack Compose

Android's modern UI toolkit is built on Kotlin, allowing you to create UI quickly with powerful and intuitive APIs.

[Learn more](#)



Structured concurrency

Kotlin coroutines streamline asynchronous programming, making common tasks like network calls and database updates simple and performant.

Kotlin

Kotlin Basics

- A Kotlin program requires a `main` function as the entry point of the program.
- To define a function in Kotlin, use the `fun` keyword

```
fun name ( parameters ) : return type {  
    body  
}
```

Kotlin Basics

- Use the `val` keyword to define a variable that is read-only where the value cannot change once it's been assigned.
- Use the `var` keyword to define a variable that is mutable or changeable.
- In Kotlin, it's preferred to use `val` over `var` when possible.
- To declare a variable, start with the `val` or `var` keyword. Then specify the variable name, data type, and initial value. For example: `val count: Int = 2`.
- With type inference, omit the data type in the variable declaration if an initial value is provided.
- Some common basic Kotlin data types include: `Int`, `String`, `Boolean`, `Float`, and `Double`.

Android Studio - Project Structure

- **manifests**

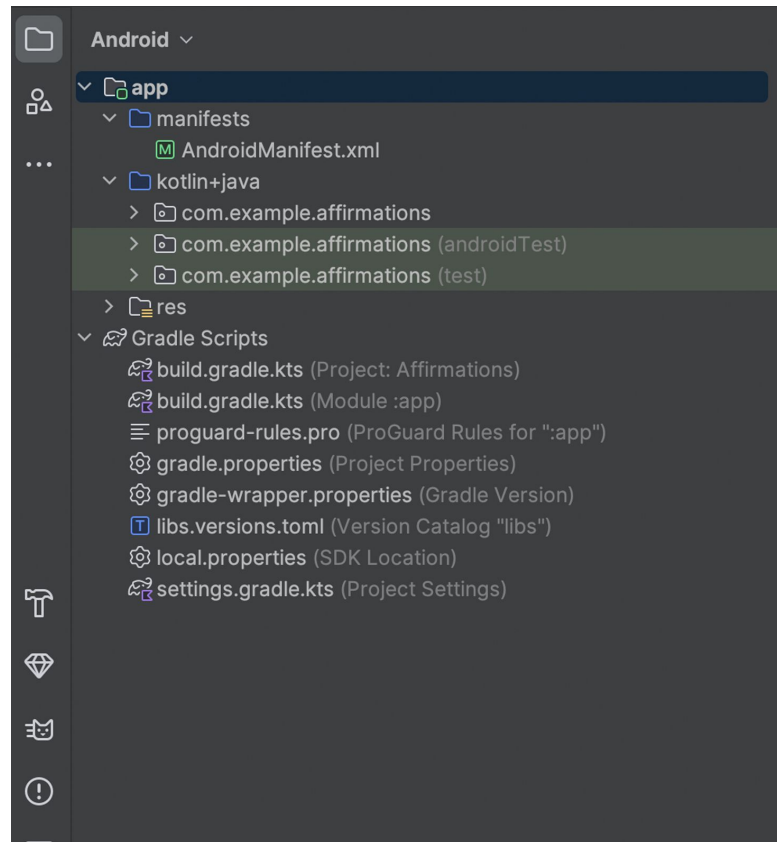
Contains the AndroidManifest.xml file.

- **kotlin+java**

Contains the Kotlin and Java source code files, including JUnit test code.

- **res**

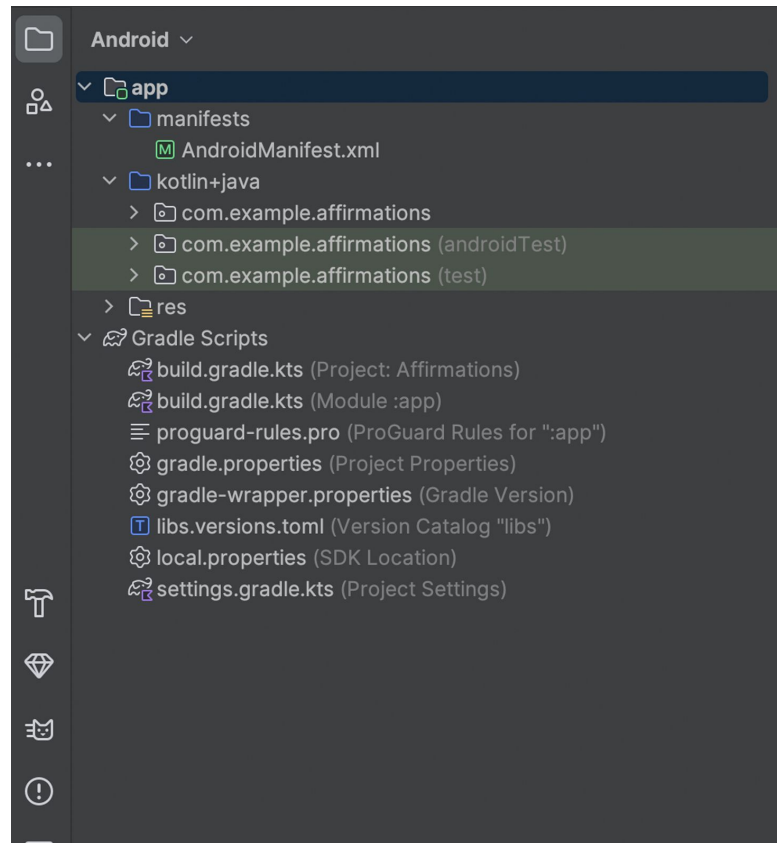
Contains all non-code resources such as UI strings and bitmap images.



Android Studio - Gradle

Build system runs as an integrated tool from the **Android Studio**

- Customize, configure, and extend the build process.
- Create multiple APKs for your app with different features, using the same project and modules.
- **Reuse code and resources across source sets.**
- **Manage dependencies**



Creating Your First App!

- Today we will do it together

Create App

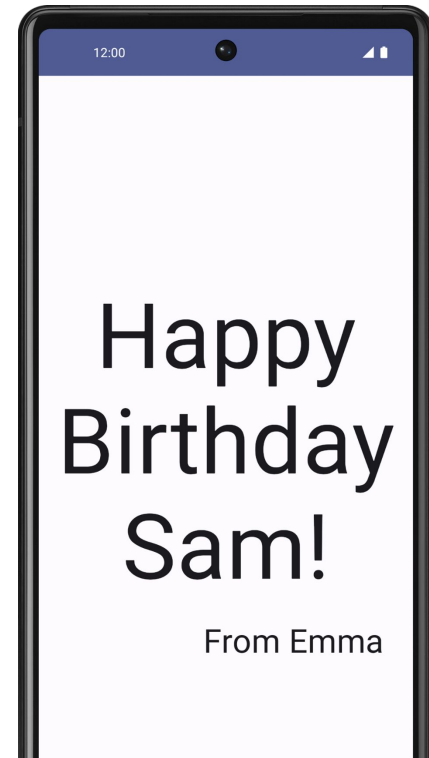
- [https://developer.android.com/codelabs/basic-and-roid-kotlin-compose-first-app#7](https://developer.android.com/codelabs/basic-android-kotlin-compose-first-app#7)

Deploy (recommend use your device)

- <https://tinyurl.com/deployCM>

First Interface

- <https://tinyurl.com/textComposablesCM>



-Goal for today-

Android Interfaces

Old:

- XML interfaces
- Mostly Static
- Harder to
 - Maintain
 - Scale
 - Interactive

```
<TextView  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Hello World!" />
```

Android Interfaces

Now:

- Jetpack composer
- Flexible
- Reactive

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name!")
}
```

JetPack Compose

Write interfaces using Jetpack Compose, generates the appropriate screens.

	XML	Jetpack Compose
Syntax	Verbose, declarative and too much text	Declarative, programmatic and concise
Language	Java or Kotlin	Kotlin only
Code structure	Each layout has a separate XML file. Sometimes complex and hard to maintain	Inline UI Kotlin code and easy to maintain
Reactivity	Non-reactive	Reactive
Learning curve	Easy to learn	Easier to learn than XML
Customization	Customizable	Highly customizable than XML
Adoption	Widely used in old/earlier projects	Widely used in recent projects

JetPack Compose

```
@Composable
fun JetpackCompose() {
    Card {
        var expanded by remember { mutableStateOf(false) }
        Column(Modifier.clickable { expanded = !expanded }) {
            Image(painterResource(R.drawable.jetpack_compose))
            AnimatedVisibility(expanded) {
                Text(
                    text = "Jetpack Compose",
                    style = MaterialTheme.typography.bodyLarge,
                )
            }
        }
    }
}
```



Jetpack
Compose

JetPack Compose

Prefix character: @

Annotation

@Composable

```
fun Greeting(name: String, modifier: Modifier) {}
```

Function declaration

Debug your app

- Logcat helps you debug your app by displaying logs from your device in real time

More Details:

<https://developer.android.com/studio/debug>

<https://developer.android.com/studio/debug/logcat>

Debug - Using Logcat

```
private const val TAG = "MainActivity"
```



4. In the `onCreate()` method, just after the call to `super.onCreate()`, add the following line:

```
Log.d(TAG, "onCreate Called")
```



Summary

- Android Studio Overview
- Kotlin & Composable Getting Started
- Running your first app
- Logcat & Debug

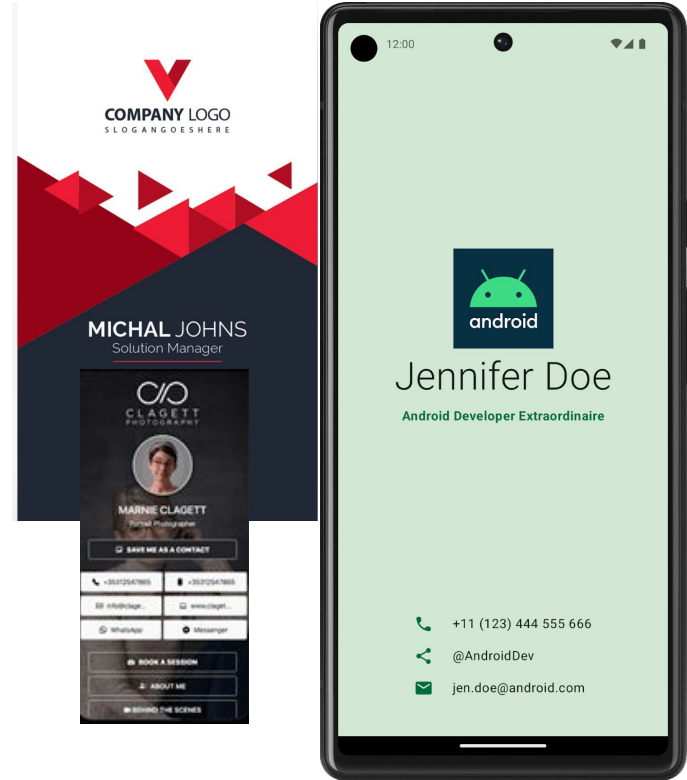
Assignment #1 - Favorite Travel Badge

Create your **unique** (in **content** and **LAYOUT**) one screen digital travel badge.

It has to use multiple Text and Image Composable and be in a different configuration than the business card examples.

Include information about your favorite place to travel (e.g., a generic landscape/city image, the destination name, or current weather details like). It should include more elements than just these examples.

Codelabs for example: <https://tinyurl.com/cmlab1asign>



Next Class

Build your app UI

Useful Links

Kotlin fundamentals:

<https://developer.android.com/courses/pathways/android-basics-compose-unit-2-pathway-1>